

UNIVERSITÉ DE TOULOUSE

Faculté Sciences et Ingénierie
Licence Informatique

ANNÉE UNIVERSITAIRE 2025 / 2026

CAHIER DES CHARGES PRÉVISIONNEL

Bureau d'études - CC1

**Simulation d'ordonnancement sur cloud
avec recommandations de clusters**

Encadrant	Eyvaz Ahmadzada
Laboratoire	IRIT
Étudiants	Birane Ndiaye Mathis Fontanié Emmanuel Denis Tom Tassin
Outils envisagés	Python, Pandas, Numpy, Matplotlib

Mars 2026

1. Introduction et contexte du projet

Le projet que nous présentons ici porte sur la simulation d'un ordonnancement de jobs sur des clusters cloud hétérogènes, en s'appuyant sur des recommandations énergétiques déjà produites par un système de recherche. Aujourd'hui, les centres de calcul consomment énormément d'énergie, et cette question est devenue incontournable, autant sur le plan économique qu'environnemental. Dans ce contexte, mieux répartir les jobs entre plusieurs clusters pourrait permettre de réduire une partie de cette consommation, à condition d'avoir les bons outils pour estimer l'impact de telles décisions.

Le sujet s'inscrit dans un cadre assez concret : pour chaque job déjà exécuté par le passé, un système de recommandation propose une liste ordonnée de clusters jugés plus intéressants du point de vue énergétique. Ce système fournit aussi des estimations sur le temps d'exécution, la courbe de puissance et la consommation d'énergie attendue sur les clusters recommandés. L'idée n'est donc pas de créer un nouveau système de recommandation, mais plutôt d'utiliser des recommandations déjà calculées pour observer ce qui se serait passé si elles avaient été suivies.

Notre objectif est de construire une simulation à partir de données historiques. Concrètement, on repart d'exécutions réelles déjà observées, puis on rejoue ces jobs en supposant, selon différents scénarios, qu'ils auraient pu être envoyés vers d'autres clusters. L'idée est de comparer la situation réellement observée avec plusieurs situations simulées, pour estimer les gains énergétiques possibles tout en gardant un œil sur les effets secondaires (temps d'exécution, disponibilité des ressources, etc.).

2. Analyse du besoin et reformulation du sujet

Si on reformule le besoin de façon simple, il s'agit de concevoir un programme capable de rejouer des exécutions passées en remplaçant, quand c'est pertinent, le cluster réellement utilisé par un cluster recommandé. Pour chaque job, la simulation devra parcourir la liste des clusters proposés, vérifier si l'un d'eux est disponible au moment où le job arrive, puis choisir un cluster selon les règles du scénario testé.

Le sujet ne se limite pas à faire tourner des scripts. Il demande surtout de définir une méthode claire et paramétrable. Il faut donc prévoir un moteur de simulation compréhensible, avec des hypothèses bien explicitées, et des sorties exploitables pour l'analyse. Le programme devra être capable de tester plusieurs scénarios de comportement utilisateur, afin de représenter différents niveaux d'adhésion aux recommandations.

Un point important à retenir : un cluster recommandé n'est pas forcément disponible immédiatement. Il ne suffit donc pas de prendre automatiquement la première recommandation. Il faut intégrer des règles réalistes, par exemple tester le premier cluster de la liste, attendre un certain temps si besoin, puis passer au suivant si l'attente devient trop importante. C'est cette partie qui rend la simulation plus crédible.

3. Données manipulées et hypothèses de travail

Pour faire tourner la simulation, on a besoin de deux grandes catégories de données. D'abord, les données historiques d'exécution des jobs : identifiant du job, date de soumission, cluster réellement utilisé, durée observée, etc. Ensuite, les sorties du système de recommandation : pour chaque job, une liste ordonnée de clusters recommandés avec les estimations associées en temps, puissance et énergie.

À ce stade, quelques hypothèses de travail sont nécessaires. On suppose que les estimations fournies par le système de recommandation sont suffisamment fiables pour servir de base à la simulation. On considère aussi que l'ordre des clusters recommandés reflète une préférence énergétique, c'est-à-dire que le premier cluster de la liste est normalement le plus intéressant. Enfin, si la disponibilité n'est pas directement fournie dans les données, il faudra l'approcher par un modèle simplifié, en suivant par exemple l'occupation simulée des clusters au fil du temps.

On garde évidemment en tête les limites de l'exercice. Une simulation reste une approximation de la réalité, surtout quand toutes les informations du système réel ne sont pas accessibles. Le comportement des utilisateurs peut être plus complexe qu'un simple « oui ou non » face à une recommandation. Mais ce type d'outil reste utile pour estimer des tendances globales et comparer plusieurs politiques d'ordonnement dans un cadre contrôlé.

4. Méthodologie envisagée pour la simulation

L'idée est de rejouer les jobs dans l'ordre chronologique de soumission. Pour chaque job, le programme récupère la liste des clusters recommandés, puis applique une série d'étapes pour décider où il est envoyé dans la simulation. Si le premier cluster recommandé est disponible, le job lui est affecté directement. Sinon, la simulation attend un peu ou teste le cluster suivant, selon les règles du scénario en cours.

Le moteur de simulation devra donc maintenir un état des clusters au cours du temps. Une manière simple de procéder serait de suivre, pour chaque cluster, les jobs qui lui ont déjà été affectés et leur durée estimée. Quand un nouveau job arrive, on peut alors savoir si le cluster est libre immédiatement, s'il faut attendre un peu, ou s'il vaut mieux essayer la recommandation suivante.

Plusieurs paramètres devront être configurables pour rendre la simulation exploitable : la probabilité de suivi de la recommandation, le temps d'attente maximal autorisé, ou encore une graine aléatoire pour pouvoir reproduire exactement les mêmes expériences. Cette approche permettra de tester facilement différents scénarios sans toucher à l'architecture du programme.

Le développement sera fait en Python. Pandas servira à charger et manipuler les données, Numpy pour certains calculs et la gestion de l'aléatoire, et Matplotlib pour produire des graphiques comparatifs lisibles.

5. Scénarios à étudier

Le premier scénario, imposé par le sujet, est celui où tous les utilisateurs suivent les recommandations. La simulation essaie systématiquement d'exécuter les jobs sur les clusters recommandés, en tenant compte de leur disponibilité. C'est une situation idéale du point de vue de l'adoption du système.

Le deuxième scénario repose sur un comportement aléatoire : un utilisateur peut suivre ou non la recommandation sans règle fixe. Ça permet de modéliser une situation plus incertaine, où l'adhésion existe mais reste partielle et variable.

Le troisième scénario est une version paramétrable : les utilisateurs suivent la recommandation avec une probabilité choisie à l'avance (30 %, 50 %, 80 %...). C'est particulièrement intéressant car ça permet de voir comment les résultats évoluent quand l'acceptation des recommandations augmente progressivement.

6. Résultats attendus et critères d'évaluation

Le critère principal sera la consommation énergétique totale simulée. On veut savoir si le fait d'utiliser les clusters recommandés permet réellement de réduire l'énergie consommée par rapport à la situation historique. C'est le cœur de l'analyse.

On compte aussi suivre d'autres indicateurs pour ne pas se limiter à un seul point de vue : le temps d'exécution moyen ou total des jobs (pour vérifier qu'un gain énergétique ne se fait pas au prix de performances dégradées), la répartition des jobs sur les clusters, le nombre de recommandations effectivement suivies, et les cas où un cluster recommandé n'était pas disponible.

Au final, le projet devra produire des scripts réutilisables, des paramètres faciles à modifier, des tableaux de résultats et plusieurs visualisations. Ces sorties seront utiles pour le rapport final mais aussi pour la soutenance.

7. Organisation du travail dans le groupe

Le travail sera réparti entre les quatre membres du groupe, en gardant un fonctionnement collectif. Emmanuel Denis prend en charge la coordination générale du projet, l'harmonisation des rapports et une partie de la conception de la simulation. Birane Ndiaye se concentre davantage sur l'analyse et la préparation des données, pour comprendre leur structure et mettre en place une base de travail propre.

Mathis Fontanié s'occupe plus particulièrement de la logique de simulation : scénarios utilisateurs, règles de disponibilité, choix final des clusters. Tom Tassin contribue surtout à l'exploitation expérimentale, l'analyse comparative des résultats, la création de graphiques et la documentation technique. Évidemment, cette répartition reste prévisionnelle et pourra bouger selon les besoins.

Pour garder le travail cohérent, on prévoit d'utiliser un dépôt commun pour le code, avec des échanges réguliers pour faire le point. Les décisions importantes seront prises ensemble pour éviter que chaque partie soit développée de manière isolée.

8. Échéancier prévisionnel

Dans un premier temps, on prévoit une phase de compréhension du sujet et des données. Cette étape sert à préciser les hypothèses, repérer les informations réellement disponibles et mieux cerner les contraintes du problème. C'est indispensable pour ne pas construire une simulation sur des bases floues.

Ensuite vient la phase de conception : poser clairement le fonctionnement du simulateur, le modèle de disponibilité, la gestion des scénarios, le choix des paramètres et l'organisation du code. Une fois cette base posée, on passe à l'implémentation d'une première version capable de charger les données et de faire tourner un scénario simple.

La suite consistera à enrichir cette première version avec tous les scénarios demandés, puis à lancer des expériences comparatives. Enfin, une dernière phase sera consacrée à l'analyse des résultats, la mise au propre du code, la rédaction du rapport final et la préparation de la soutenance. Cet échéancier reste volontairement souple et sera affiné dans le bilan de mi-parcours.

Annexe – Répartition prévisionnelle des phases

Phase	Contenu	Période estimée
Phase 1	Compréhension du sujet et des données	2 mars – 15 mars
Phase 2	Conception du simulateur	16 mars – 5 avril
Phase 3	Première version fonctionnelle	6 avril – 19 avril
Phase 4	Ajout des scénarios et tests	20 avril – 10 mai
Phase 5	Analyse, rapport final, soutenance	À partir du 11 mai

9. Conclusion

Ce projet a pour but d'étudier, par la simulation, l'impact d'un ordonnancement davantage guidé par des recommandations énergétiques sur des clusters cloud hétérogènes. À partir de données historiques et de résultats déjà produits par un système de recommandation, on veut construire un outil capable de comparer plusieurs comportements utilisateurs et d'évaluer leurs conséquences.

Le travail attendu mélange donc plusieurs dimensions : compréhension des données, modélisation du problème, développement d'un simulateur et analyse des résultats. Ce rapport de CC1 pose avant tout une base claire pour la suite. Il fixe notre compréhension du sujet, notre méthode de travail et une première organisation collective, qui seront ensuite affinées au fur et à mesure de l'avancement.